

# fn make\_clamp

Sílvia Casacuberta (@silviacasac)

This proof resides in “contrib” because it has not completed the vetting process.

Proves soundness of fn make\_clamp in [mod.rs at commit 0db9c6036](#) (outdated<sup>1</sup>).

## 1 Hoare Triple

### Precondition

#### Compiler-verified

- Type TA must have trait ProductOrd.
- Type M must have trait DatasetMetric.

#### Caller-verified

None

### Pseudocode

```
1 def make_clamp(  
2     input_domain: VectorDomain[AtomDomain[TA]],  
3     input_metric: M,  
4     bounds: tuple[TA, TA]  
5 ): #  
6     input_domain.element_domain.assert_non_null() #  
7  
8     # clone to make it explicit that we are not mutating the input domain  
9     output_row_domain = input_domain.element_domain.clone()  
10    output_row_domain.bounds = Bounds.new_closed(bounds)  
11  
12    def clamper(value: TA) -> TA: #  
13        return value.total_clamp(bounds[0], bounds[1])  
14  
15    return make_row_by_row_fallible( #  
16        input_domain,  
17        input_metric,  
18        output_row_domain,  
19        clamper  
20    )
```

### Postcondition

**Theorem 1.1.** For every setting of the input parameters (input\_domain, input\_metric, bounds) to make\_clamp such that the given preconditions hold, make\_clamp raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

<sup>1</sup>See new changes with `git diff 0db9c6036..f0a8758 rust/src/transformations/clamp/mod.rs`

1. (Data-independent runtime errors). For every pair of members  $x$  and  $x'$  in `input_domain`, `invoke(x)` and `invoke(x')` either both return the same error or neither return an error.
2. (Appropriate output domain). For every member  $x$  in `input_domain`, `function(x)` is in `output_domain` or raises a data-independent runtime error.
3. (Stability guarantee). For every pair of members  $x$  and  $x'$  in `input_domain` and for every pair  $(d\_in, d\_out)$ , where  $d\_in$  has the associated type for `input_metric` and  $d\_out$  has the associated type for `output_metric`, if  $x, x'$  are  $d\_in$ -close under `input_metric`, `stability_map(d_in)` does not raise an error, and `stability_map(d_in) = d_out`, then `function(x), function(x')` are  $d\_out$ -close under `output_metric`.

**Lemma 1.2.** The invocation of `make_row_by_row_fallible` (line 15) satisfies its preconditions.

*Proof.* The preconditions of `make_clamp` and pseudocode definition (line 5) ensure that the type preconditions of `make_row_by_row_fallible` are satisfied. The remaining preconditions of `make_row_by_row_fallible` are:

- `row_function` has no side-effects.
- If the input to `row_function` is a member of `input_domain`'s row domain, then the output is a member of `output_row_domain`.

The first precondition is satisfied by the definition of `clammer` (line 12) in the pseudocode.

For the second precondition, assume the input is a member of `input_domain`'s row domain. Therefore, by 6, the input is non-null. In addition, since `Bounds.new_closed` did not raise an exception, then by the definition of `Bounds.new_closed`, the bounds are non-null. Thus, by the definition of `ProductOrd`, the preconditions of `total_clamp` are satisfied, so the output is within the bounds. Therefore, the output is a member of `output_row_domain`.  $\square$

We now prove the postcondition of `make_clamp`.

*Proof.* By 1, the preconditions of `make_row_by_row_fallible` are satisfied. Thus, by the definition of `make_row_by_row_fallible`, the output is a valid transformation.  $\square$